While working with `Polygon` you do not need to be aware of all its descendants, because anything in the table is by definition a polygon, but when working with subclasses Active Record needs to be able to enumerate the types it is looking for. Let's see an example.

`Rectangle.all` only loads rectangles by adding a type constraint to the query:

```
SELECT "polygons".* FROM "polygons"
WHERE "polygons"."type" IN ("Rectangle")
```

Let's introduce now a subclass of `Rectangle`:

```
# app/models/square.rb
class Square < Rectangle
end
```

`Rectangle.all` should now return rectangles **and** squares:

```
SELECT "polygons".* FROM "polygons"
WHERE "polygons"."type" IN ("Rectangle", "Square")
```

But there's a caveat here: How does Active Record know that the class `Square` exists at all?

Even if the file `app/models/square.rb` exists and defines the `Square` class, if no code yet used that class, `Rectangle.all` issues the query

```
SELECT "polygons".* FROM "polygons"
WHERE "polygons"."type" IN ("Rectangle")
```

That is not a bug, the query includes all *known* descendants of `Rectangle`.

A way to ensure this works correctly regardless of the order of execution is to load the leaves of the tree by hand at the bottom of the file that defines the root class:

```
# app/models/polygon.rb
class Polygon < ActiveRecord::Base
end
require_dependency 'square'
```

Only the leaves that are **at least grandchildren** need to be loaded this way. Direct subclasses do not need to be preloaded. If the hierarchy is deeper, intermediate classes will be autoloaded recursively from the bottom because their constant will appear in the class definitions as superclass.

10.3 Autoloading and `require`

Files defining constants to be autoloaded should never be `required`:

```
require 'user' # DO NOT DO THIS

class UsersController < ApplicationController
  ...
end
```

There are two possible gotchas here in development mode:

1. If `User` is autoloaded before reaching the `require`, `app/models/user.rb` runs again because`load` does not update `$LOADED_FEATURES`.
2. If the `require` runs first Rails does not mark `User` as an autoloaded constant and changes to `app/models/user.rb` aren't reloaded.

Just follow the flow and use constant autoloading always, never mix autoloading and `require`. As a last resort, if some file absolutely needs to load a certain file use `require_dependency` to play nice with constant autoloading. This option is rarely needed in practice, though.

Of course, using `require` in autoloaded files to load ordinary 3rd party libraries is fine, and Rails is able to distinguish their constants, they are not marked as autoloaded.

10.4 Autoloading and Initializers

Consider this assignment in `config/initializers/set_auth_service.rb`:

```
AUTH_SERVICE = if Rails.env.production?
  RealAuthService
else
```